

Construcción de un Mini-Gestor de Base de Datos: WackDB

Curso: Base de Datos II - 2026-A

Autores:

Hañari Cutipa, Cesar Alejandro

Montañez Pacco, Roni Ezequiel

Rivera Torres, Jose Alberto

Índice

1. Aspectos Generales	2
1.1. Descripción del problema a resolver	2
1.2. Objetivos específicos	2
1.3. Alcance del proyecto y Limitaciones	2
1.4. Lenguaje de programación y herramientas utilizadas	2
2. Diseño de la Solución (Arquitectura)	3
2.1. Módulo de Almacenamiento (Storage Manager)	3
2.2. Módulo de Gestión de Memoria (Buffer Manager)	5
2.3. Módulo de Métodos de Acceso	6
2.4. Módulo de Procesamiento de Consultas	8
3. Implementación	10
3.1. Storage Manager	10
3.2. Buffer Manager	11
3.3. Métodos de Acceso (B+ Tree)	12
3.4. Procesadores de Consultas	14
4. Análisis y Resultados	16
4.1. Funcionalidades Probadas	16
4.2. Análisis Comparativo de Rendimiento	16
4.3. Métricas de Aciertos del Buffer Manager (Hit Rate)	18
5. Conclusiones y Recomendaciones	19
6. Anexos	19
6.1. Repositorio	19
6.2. Evidencia de Trabajo Grupal (Reuniones de Coordinación)	20

1. Aspectos Generales

1.1. Descripción del problema a resolver

La arquitectura interna de los sistemas gestores de bases de datos (SGBD) comerciales modernos suele presentar una alta complejidad, lo que dificulta su estudio analítico en entornos académicos. El presente trabajo describe el diseño e implementación de WackDB, un SGBD relacional, ligero y modular. Este proyecto busca resolver el desafío de construir un motor de almacenamiento y consultas desde cero, permitiendo una comprensión profunda de la persistencia de datos, la gestión de la memoria secundaria y la manipulación de estructuras binarias.

1.2. Objetivos específicos

- Diseñar una arquitectura modular inspirada en PostgreSQL.
- Implementar un almacenamiento persistente mediante archivos binarios.
- Garantizar la seguridad de memoria y eficiencia operativa utilizando Rust.

1.3. Alcance del proyecto y Limitaciones

Alcance: El proyecto abarca la implementación de los siguientes módulos principales:

- **Storage Manager:** Gestión de almacenamiento persistente mediante archivos binarios y páginas de tamaño fijo (8KB) utilizando el diseño de **Slot Directory** para el manejo eficiente de registros, soportando eliminación lógica.
- **Buffer Manager:** Módulo concurrente que administra un **Buffer Pool** en memoria principal, implementando obligatoriamente la política de reemplazo estricta **LRU (Least Recently Used)** para el desalojo seguro de páginas.
- **Índices (Métodos de Acceso):** Construcción conceptual y práctica de un índice de árbol B+ (**B+ Tree**) persistente para la indexación de tablas, incluyendo rebalanceo y fusión de nodos (**Merging**).
- **Procesador de Consultas:** Motor de ejecución iterativo basado en el Modelo Volcano, con soporte para operadores relacionales (Scan, Select, Project, Join, Sort). Incluye la implementación de un algoritmo de unión (**Nested Loop Join**), ordenamiento en disco mediante **External Merge Sort**, y una optimización heurística básica encargada de elegir dinámicamente entre un Escaneo Secuencial (Scan) o el uso del Índice (B+ Tree) según la disponibilidad de claves primarias.

Limitaciones: Por diseño académico, este motor simplificado no incluye características avanzadas de SGBD comerciales como control de concurrencia multiversión (MVCC), transacciones distribuidas, soporte para Vistas (Views) o Triggers.

1.4. Lenguaje de programación y herramientas utilizadas

- **Lenguaje Base:** Rust, seleccionado por sus estrictas garantías de seguridad de memoria (*borrow checker*), control explícito sobre la concurrencia y ausencia de recolección de basura.
- **Gestor de Dependencias:** Cargo, utilizado para la gestión del espacio de trabajo (*workspace*), compilación y ejecución de pruebas.

2. Diseño de la Solución (Arquitectura)

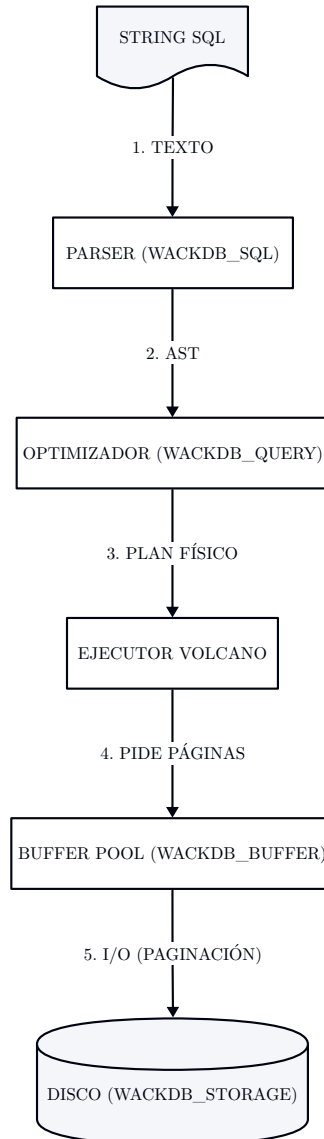


Figura 1: Visión global de la arquitectura modular y el ciclo de vida de una consulta.

2.1. Módulo de Almacenamiento (Storage Manager)

El componente `wackdb_storage` y `wackdb_page` conforman la capa inferior y fundamental del motor de base de datos, actuando como la interfaz directa con el sistema de archivos del sistema operativo. Se ha diseñado una arquitectura fuertemente tipada basada en páginas de tamaño fijo de 8 KB (8192 bytes). Esta decisión arquitectónica no es arbitraria: se alinea con los estándares de la industria (como PostgreSQL) para emparejarse con el tamaño de bloque físico de los discos SSD/HDD y las páginas de memoria virtual del kernel, minimizando así la amplificación de escritura (Write Amplification) y optimizando las operaciones de Entrada/Salida (I/O).

Para abordar el problema clásico de la fragmentación de datos al almacenar registros de longitud variable (por ejemplo, tipos **VARCHAR**), se ha implementado el patrón de diseño **Slotted Pages** (Páginas Ranuradas). Este modelo garantiza un acceso indirecto a las tuplas en tiempo constante $O(1)$ sin requerir el desplazamiento físico de registros en la memoria cuando se producen actualizaciones o eliminaciones lógicas, facilitando futuras operaciones de compactación o **Vacuum**.

A continuación se visualiza la estructura del contenedor físico, la cual es proyectada directamente a nivel de bytes mediante directivas `#[repr(C, align(8))]` en Rust para garantizar la correcta alineación de memoria y prevenir faltas de segmentación (Segmentation Faults) en arquitecturas estrictas:

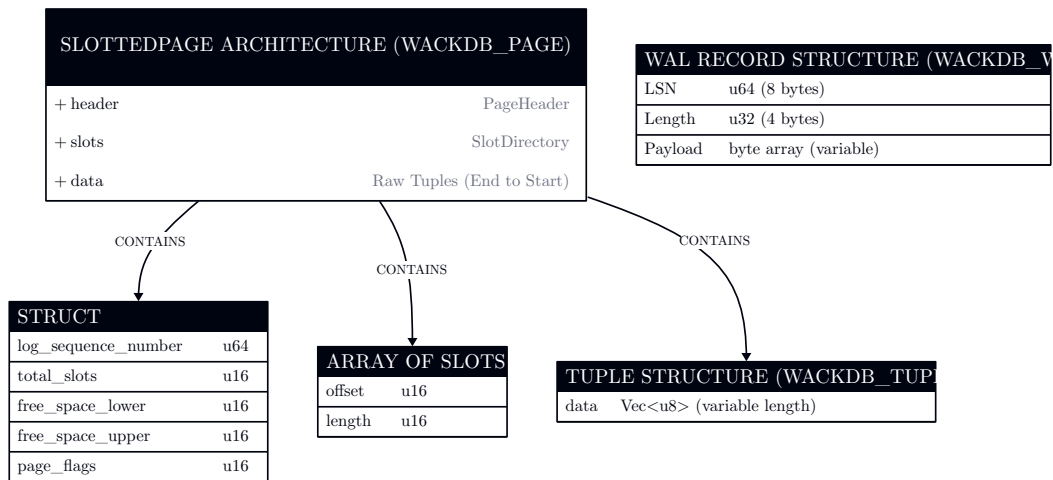


Figura 2: Arquitectura física de una Página Ranurada.

Internamente, la página se rige por la colisión de dos punteros que consumen el espacio libre desde extremos opuestos:

- **Page Header:** Metadatos críticos protegidos con un puntero `free_space_lower` que inicia inmediatamente después de la cabecera y **crece hacia abajo** conforme se añaden descriptores al directorio.
- **Slot Directory:** Un arreglo de descriptores físicos (`PageSlot`) de cada registro insertado. Cada slot actúa como un apuntador interno directo hacia los bytes exactos de la tupla.
- **Data Region:** La zona apuntada por el `free_space_upper`, el cual inicia al final de la página y **crece hacia arriba**. Cada vez que se inserta una tupla, se resta el tamaño del registro para acomodar los bytes de longitud variable al final físico del bloque.

TUPLELAYOUT	
+ TupleHeader	Metadatos (xmin, xmax)
+ NullBitmap	Mapa de bits (Opcional)
+ FixedData	Enteros (i32), Booleanos
+ VarData	Cadenas Varchar y Blobs

Figura 3: Estructura serializada de los registros (Tuplas) almacenados en la Data Region.

2.2. Módulo de Gestión de Memoria (Buffer Manager)

El componente `wackdb_buffer` expone el `BufferPoolManager`, responsable de realizar la transferencia de páginas entre el disco y la RAM. El sistema reserva una cantidad predefinida de contenedores lógicos o Frames en memoria principal.

Cada uno de estos Frames está estrictamente controlado por un descriptor físico, que integra la caché pura (el buffer de bytes) junto con su meta-estado. Su implementación en Rust emplea tipos atómicos para evitar colisiones de concurrencia de datos (Race Conditions).

Se gestionan los metadatos de caché a través de las siguientes primitivas de sincronización:

- **pin_count** (Contador de Pines): Un primitivo atómico (`AtomicUsize`) que actúa como un bloqueo lógico. Cuando un hilo de ejecución solicita acceso a una página, esta es «fijada» (pinned) en la memoria. El sistema de reemplazo tiene estrictamente prohibido desalojar cualquier marco lógico cuyo `pin_count > 0`, garantizando que las estructuras en memoria no desaparezcan a mitad de una transacción.
- **is_dirty** (Bit de Modificación): Implementado como un `AtomicBool`, optimiza las escrituras al disco siguiendo una política **Write-Back** (Escritura Diferida). Solo si este bit está encendido, el gestor sabrá que la página en la RAM difiere de su versión en disco duro y requerirá un `flush` costoso de Entrada/Salida. De lo contrario, se sobrescribe en la memoria impunemente.

Dado que la memoria RAM es finita y significativamente más pequeña que el almacenamiento secundario, se ha integrado un algoritmo **LRUReplacer** (Least Recently Used). Este componente actúa de árbitro cuando el `Buffer Pool` alcanza su capacidad tope, rastreando dinámicamente el orden cronológico de acceso. Al presentarse un fallo de caché (Cache Miss), el LRU identifica la página menos usada (la que lleva más tiempo sin ser leída ni modificada) y ordena su desalojo, logrando mantener los nodos más frecuentados (como la raíz del B+ Tree) perpetuamente cargados en la memoria rápida.

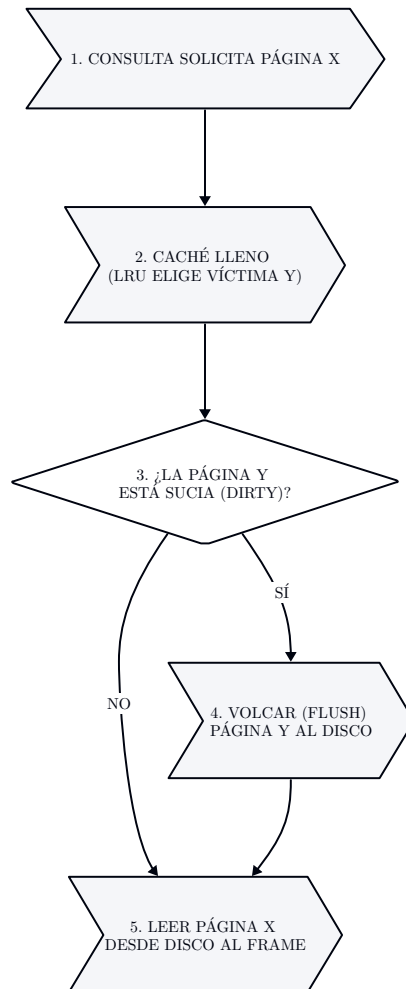


Figura 4: Flujo de decisión algorítmico del LRU Replacer para el desalojo de caché.

2.3. Módulo de Métodos de Acceso

Para evitar escaneos secuenciales costosos, se implementó un Índice de Árbol B+ (`wackdb_btree`) que interactúa directamente con el Buffer Pool. Este índice está optimizado para páginas de tamaño fijo (8 KB) mediante el uso de la serialización en memoria y no reside completamente en RAM.

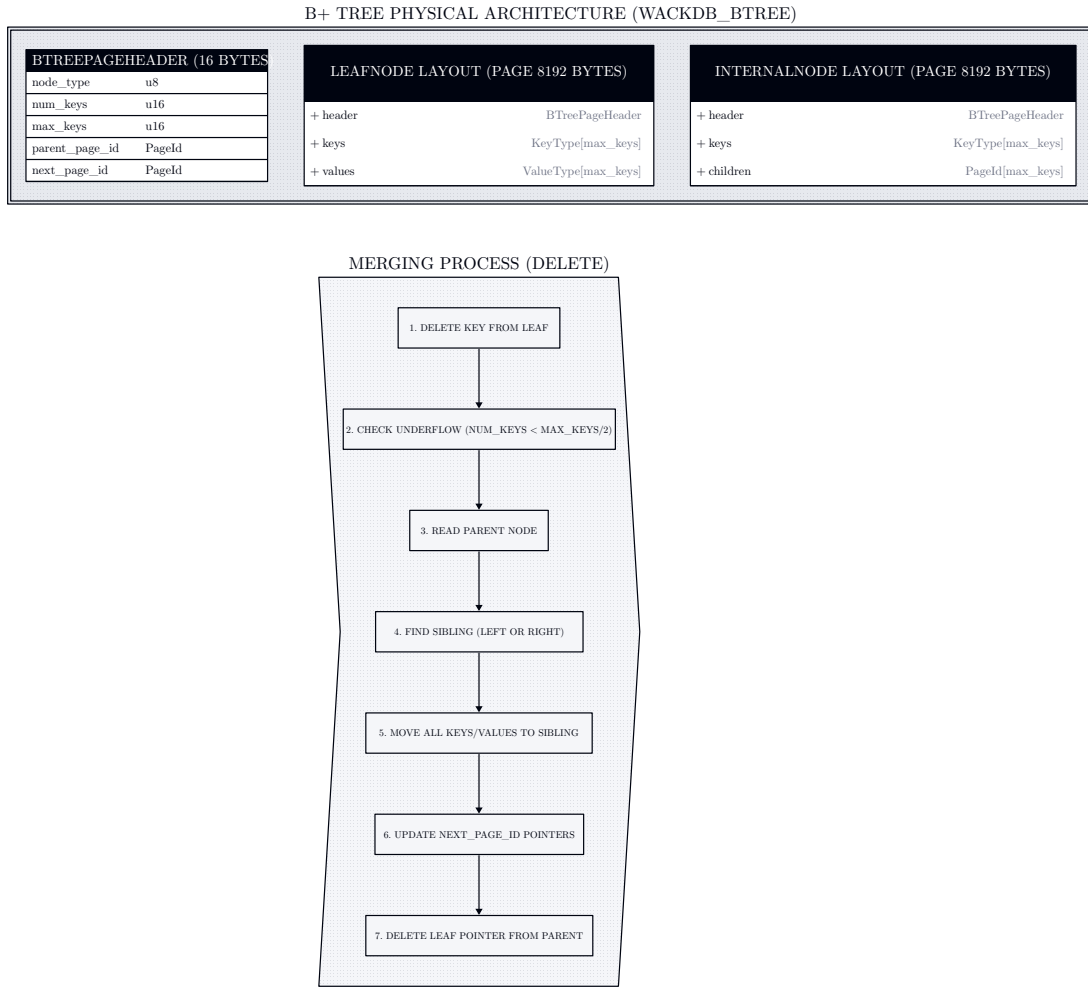


Figura 5: Arquitectura del Árbol B+ y sus Nodos.

Para maximizar el **Fan-out** (Factor de Ramificación) y minimizar la profundidad del árbol, se han abstraído dos estructuras de nodos independientes y mutuamente excluyentes:

- **Internal Nodes (Nodos Internos):** Diseñados estrictamente como nodos de enrutamiento. Almacenan exclusivamente valores de llaves (**keys**) y punteros lógicos hacia otras páginas (**PageId**). Al no contener la carga útil de los datos, un solo bloque de 8 KB puede albergar miles de punteros, permitiendo que el árbol escale masivamente manteniendo una altura logarítmica casi plana ($O(\log_{b(N)})$).
- **Leaf Nodes (Nodos Hoja):** La base de la estructura. Almacenan las llaves de búsqueda emparejadas con sus identificadores físicos finales (**CTID**). El **CTID** (Item Pointer) actúa como el ancla entre la estructura lógica del índice y la memoria física, señalando inequívocamente el ID de archivo, la página y el Slot específico en el **Storage Manager**.

Adicionalmente, se ha dotado a los nodos hoja de punteros de encadenamiento secuencial (**next_page_id**). Esto acelera de forma dramática las consultas de rango completo (Range Scans), permitiendo que el **Query Processor** barra los resultados lateralmente sin necesidad de re-atravesar el árbol desde la raíz. El sistema completo confía en la directiva `#[repr(C)]` de Rust para des-serializar la estructura directamente sobre arreglos de bytes crudos con costo $O(1)$ de deserialización.

2.4. Módulo de Procesamiento de Consultas

El procesamiento de las sentencias AST (Abstract Syntax Tree) en WackDB es delegado a un **Pipeline** regido por el clásico **Modelo Volcano** (Modelo de Iterador). La fortaleza de este paradigma radica en su comportamiento de evaluación perezosa (Lazy Evaluation).

En lugar de materializar tablas enteras en la memoria para cada operación (lo que ocasionaría desbordamientos o **Out of Memory Errors** en **datasets** masivos), cada operador relacional (Scan, Select, Project, Join) implementa un *trait* uniforme que expone el método iterativo `next() -> Option<Tuple>`. Las tuplas «fluyen» a través del grafo del plan de ejecución una por una, minimizando drásticamente la huella de memoria del motor en tiempos de ejecución (Runtime).

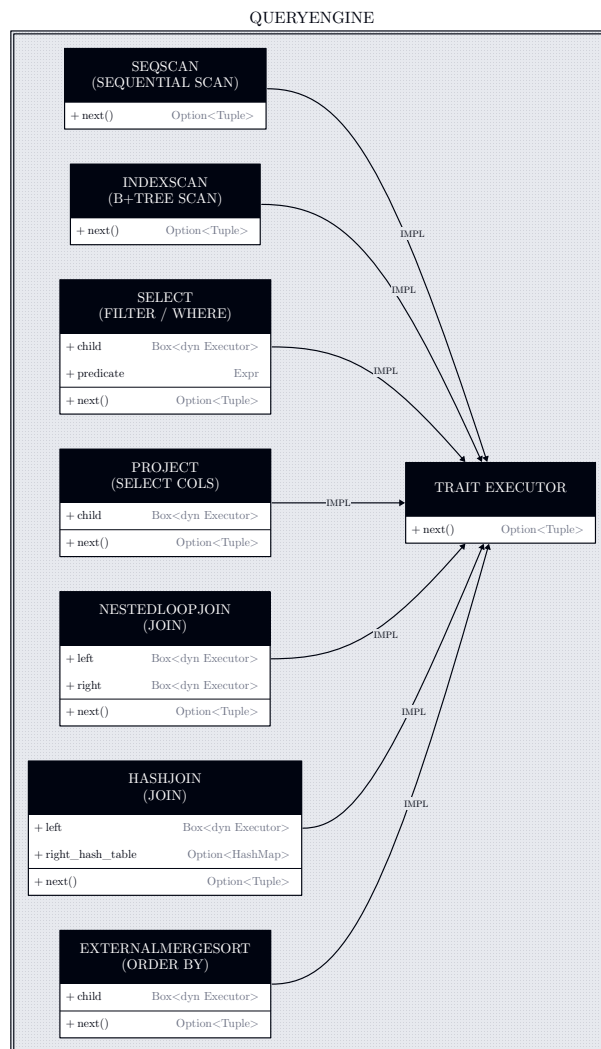


Figura 6: Arquitectura de clases y traits del Modelo Volcano (wackdb_query).



Figura 7: Ejemplo de Pipeline

- **SeqScan / IndexScan (Nodos Hoja):** Representan las fuentes de acceso físico a los datos. En lugar de cargar una tabla entera en memoria, solicitan bloques al *Buffer Manager* de manera diferida. *SeqScan* realiza un barrido secuencial $O(N)$, mientras que *IndexScan* navega el Árbol B+ y utiliza los punteros CTID para extraer tuplas directamente en $O(\log N)$.
- **Filter / Selection (Operador de Tubería):** Recibe una tupla de su operador hijo y aplica un predicado lógico (el equivalente a un sub-árbol AST de la cláusula **WHERE**). Si la expresión booleana es verdadera, la tupla se emite hacia el nodo superior; de lo contrario, se descarta instantáneamente. Actúa como un escudo que previene la propagación de datos innecesarios a través del pipeline.
- **Project (Operador de Reestructuración):** Suele ubicarse en la cúspide del plan de ejecución. Es responsable de mutar el esquema original (el *Tuple Layout* denso) para generar una nueva tupla proyectada que contenga únicamente las columnas requeridas por el usuario en el **SELECT**, disminuyendo drásticamente el tamaño del **payload** final.
- **NestedLoopJoin (Operador de Unión Binario):** Conecta dos ramas del árbol de ejecución (relación externa e interna). Su comportamiento es elástico: por cada tupla emitida por la rama izquierda, inicializa y drena la rama derecha buscando emparejamientos que satisfagan la condición. Aunque su complejidad asintótica es de $O(N \times M)$, la pereza (**laziness**) del modelo de iteradores le permite ejecutar el cruce con un uso de memoria casi constante de $O(1)$.

Al anidar dinámicamente estos operadores, el motor es capaz de resolver sentencias SQL complejas conformando un «Árbol Lógico del Plan de Ejecución» que se evalúa cooperativamente desde la raíz:

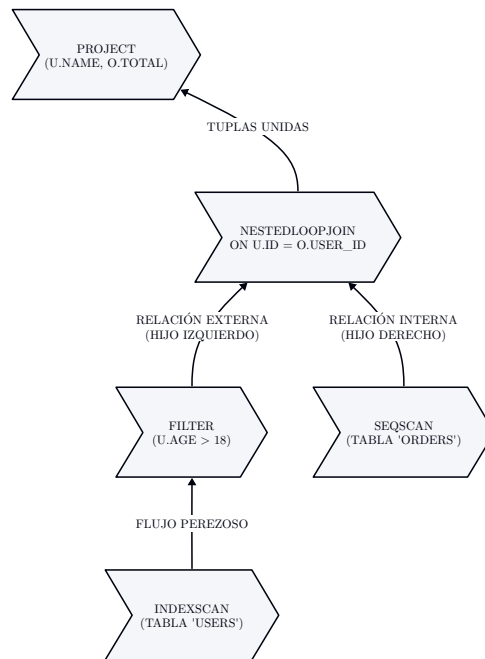


Figura 8: Estructura clásica de un Árbol del Plan de Ejecución anidando operadores Volcano.

3. Implementación

3.1. Storage Manager

La persistencia física se maneja a través del cálculo algorítmico del offset de los bytes del archivo binario. Los siguientes fragmentos exponen la lectura y escritura atómica de páginas:

Lectura de Página desde Disco:

```

pub fn read_page(&self, page_id: PageId, data: &mut [u8; PAGE_SIZE]) -> Result<(),
StorageError> {
    self.ensure_file_open(page_id.file_id)?;
    let handles = self.file_handles.lock();
    let handle = handles
        .get(&page_id.file_id)
        .ok_or(StorageError::FileError(page_id.file_id))?;

    let mut file = handle.fd.lock();
    let offset = u64::from(page_id.page_num) * (PAGE_SIZE as u64);

    let file_len = file.metadata()?.len();
    if offset >= file_len {
        data.fill(0);
        return Ok(());
    }

    file.seek(SeekFrom::Start(offset))?;
    let mut temp_buf = vec![0; PAGE_SIZE];
    let bytes_read = file.read(&mut temp_buf)?;
    data.copy_from_slice(&temp_buf);
}

```

```

    Ok(())
}

```

Escritura de Página a Disco:

```

pub fn write_page(&self, page_id: PageId, data: &[u8; PAGE_SIZE]) -> Result<(),
StorageError> {
    self.ensure_file_open(page_id.file_id)?;
    let handles = self.file_handles.lock();
    let handle = handles
        .get(&page_id.file_id)
        .ok_or(StorageError::FileError(page_id.file_id))?;

    let mut file = handle.fd.lock();
    let offset = u64::from(page_id.page_num) * (PAGE_SIZE as u64);

    file.seek(SeekFrom::Start(offset))?;
    file.write_all(data)?;

    Ok(())
}

```

3.2. Buffer Manager

La solicitud de una nueva página en el Buffer involucra lógica estricta para fijar (pin) la página en la memoria de forma segura, y desanclarla (unpin) actualizando el estado sucio (dirty):

```

pub fn new_page(&self, file_id: u32) -> Result<(usize, PageId), BufferError> {
    let page_id = self.disk_manager.allocate_page(file_id)?;

    // Evict a page if cache is full using LRU policy
    let frame_id = self.fetch_page(page_id)?;
    Ok((frame_id, page_id))
}

pub fn unpin_page(&self, page_id: PageId, is_dirty: bool) -> Result<(), BufferError>
{
    let page_table = self.page_table.read();
    if let Some(&frame_id) = page_table.get(&page_id) {
        let desc = &self.descriptors[frame_id];
        let mut count = desc.pin_count.load(Ordering::SeqCst);
        if count == 0 { return Err(BufferError::FrameNotPinned); }

        desc.pin_count.fetch_sub(1, Ordering::SeqCst);
        if is_dirty {
            desc.is_dirty.store(true, Ordering::SeqCst);
        }

        // Notify LRU that the frame is now unpinned and eligible for eviction
        self.replacer.set_pin(frame_id, false);
        Ok(())
    } else {
        Err(BufferError::PageNotFound)
    }
}

```

3.3. Métodos de Acceso (B+ Tree)

El árbol B+ interactúa con el Buffer Manager para la búsqueda e inserción.

Búsqueda en B+ Tree: Se ubica la hoja correspondiente a la llave recorriendo el árbol y se extrae el valor CTID.

```
pub fn search(&self, key: KeyType) -> Result<ValueType, BTreeError> {
    let (leaf_frame_id, curr_page_id) =
        self.find_leaf_page(key)?.ok_or(BTreeError::KeyNotFound)?;

    let page_data = self.buffer_pool.read_page(leaf_frame_id);
    let leaf_node = unsafe { &*(page_data.data.as_ptr() as *const LeafNode) };
    let num_keys = leaf_node.header.num_keys as usize;

    let found_val = leaf_node.keys()[..num_keys]
        .binary_search(&key)
        .ok()
        .map(|idx| leaf_node.values()[idx]);

    drop(page_data);
    self.buffer_pool.unpin_page(curr_page_id, false)?;
    found_val.ok_or(BTreeError::KeyNotFound)
}
```

Inserción en B+ Tree: Se determina si hay espacio y se inserta.

```
pub fn insert(&self, key: KeyType, value: ValueType) -> Result<(), BTreeError> {
    let root_id_opt = *self.root_page_id.read();

    if root_id_opt.is_none() {
        return self.create_root_and_insert(key, value);
    }

    let (leaf_frame, leaf_page_id) =
        self.find_leaf_page(key)?.ok_or(BTreeError::KeyNotFound)?;
    self.insert_into_leaf(leaf_frame, leaf_page_id, key, value)
}
```

Eliminación y Fusión (Merging) en B+ Tree: El borrado verifica el balanceo y fusiona nodos en underflow.

```
pub fn delete(&self, key: KeyType) -> Result<(), BTreeError> {
    // Locate leaf
    let (leaf_frame, leaf_page_id) =
        self.find_leaf_page(key)?.ok_or(BTreeError::KeyNotFound)?;

    // Logical deletion on the leaf and underflow detection
    let underflow = self.delete_from_leaf(leaf_frame, leaf_page_id, key)?;

    if underflow {
        // recursive merging process
        self.coalesce_or_redistribute(leaf_page_id)?;
    }
    Ok(())
}
```

Adicionalmente, se verificó el pipeline de eliminación de registros desde la consola interactiva. Primero, se listaron los registros existentes en la base de datos:

```
wackdb (wackdb_data)> select * from demo_users;
```

id	is_admin	username	status
1	false	user_1	active
2	false	user_2	active
3	false	user_3	active
4	false	user_4	active
5	true	user_5	active
6	false	user_6	active
7	false	user_7	active
8	false	user_8	active
9	false	user_9	active
10	true	user_10	active
11	false	user_11	active
12	false	user_12	active
13	false	user_13	active
14	false	user_14	active
15	true	user_15	active

```

15 row(s) returned.
Execution time: 2.216068ms

- Buffer Cache Hits: 16
- Cache Misses (Disk I/O): 0 (Hit Rate: 100.00%)
- Execution Pipeline: SeqScan(demo_users)

```

Figura 9: Consulta inicial mostrando 15 registros activos en el sistema.

Posteriormente, se ejecutó una sentencia DELETE filtrando por condición (`id < 10`). Como se observa, el **DeleteExecutor** eliminó atómicamente 9 registros y provocó volcados (flushes) en el Write-Ahead Log (WAL) para garantizar la durabilidad de los borrados lógicos. Una consulta posterior confirma que los registros desaparecieron:

```
wackdb (wackdb_data)> delete from demo_users where id < 10;
[OK] Deleted 9 rows from 'demo_users'.
Telemetry: [WAL] Log records flushed up to LSN 9
Execution time: 1.476845ms

- Buffer Cache Hits: 10
- Cache Misses (Disk I/O): 0 (Hit Rate: 100.00%)
- Execution Pipeline: SeqScan(demo_users) -> DeleteExecutor

wackdb (wackdb_data)> select * from demo_users;
```

id	is_admin	username	status
10	true	user_10	active
11	false	user_11	active
12	false	user_12	active
13	false	user_13	active
14	false	user_14	active
15	true	user_15	active

```

6 row(s) returned.
Execution time: 369.177µs

- Buffer Cache Hits: 7
- Cache Misses (Disk I/O): 0 (Hit Rate: 100.00%)
- Execution Pipeline: SeqScan(demo_users)

```

Figura 10: Eliminación exitosa de 9 registros y actualización en tiempo real de la tabla.

3.4. Procesadores de Consultas

Se implementó la unión de tablas usando **Nested Loop Join** y ordenamiento en disco con **External Merge Sort**:

Nested Loop Join:

```
impl<L: Executor, R: Executor> Executor for NestedLoopJoin<L, R> {
    fn next(&mut self) -> Result<Option<Tuple>, QueryError> {
        if self.right_tuples.is_none() {
            let mut rt = Vec::new();
            while let Some(t) = self.right.next()? {
                rt.push(t);
            }
            self.right_tuples = Some(rt);
        }

        loop {
            if self.left_tuple.is_none() {
                self.left_tuple = self.left.next()?;
                if self.left_tuple.is_none() {
                    return Ok(None);
                }
                self.right_index = 0;
            }

            let left = self.left_tuple.as_ref().unwrap();
            let right_list = self.right_tuples.as_ref().unwrap();

            while self.right_index < right_list.len() {
                let right = &right_list[self.right_index];
                self.right_index += 1;

                if (self.predicate)(left, self.left.schema(), right,
self.right.schema()) {
                    let mut joined_data = left.data.clone();
                    joined_data.extend_from_slice(&right.data);
                    return Ok(Some(Tuple { data: joined_data }));
                }
            }
            self.left_tuple = None;
        }
    }
}
```

External Merge Sort:

```
impl<E: Executor> ExternalMergeSort<E> {
    fn init(&mut self) -> Result<(), QueryError> {
        let mut buffer = Vec::new();
        let schema = self.child.schema().clone();

        while let Some(t) = self.child.next()? {
            buffer.push(t);
            if buffer.len() >= self.chunk_size {
                self.sort_buffer(&mut buffer, &schema);
                self.spill_to_disk(&mut buffer)?;
            }
        }
    }
}
```

```

    }

    if self.runs.is_empty() {
        // Fits entirely in memory
        self.sort_buffer(&mut buffer, &schema);
        self.in_memory = Some(buffer.into_iter());
    } else {
        // Spill the remaining tuples
        if !buffer.is_empty() {
            self.sort_buffer(&mut buffer, &schema);
            self.spill_to_disk(&mut buffer)?;
        }

        // Initialize active runs
        for mut file in std::mem::take(&mut self.runs) {
            file.seek(SeekFrom::Start(0))
                .map_err(|_| QueryError::Execution("Seek failed".into()))?;
            let tuple = Self::read_tuple(&mut file)?;
            self.active_runs.push((file, tuple));
        }
        self.initialized = true;
        Ok(())
    }
}

impl<E: Executor> Executor for ExternalMergeSort<E> {
    fn next(&mut self) -> Result<Option<Tuple>, QueryError> {
        if !self.initialized {
            self.init()?;
        }
        if let Some(in_mem) = &mut self.in_memory {
            return Ok(in_mem.next());
        }

        let schema = self.child.schema();
        let idx = self.sort_col_idx;
        let mut best_idx: Option<usize> = None;

        for (i, (_, tuple_opt)) in self.active_runs.iter().enumerate() {
            if let Some(t) = tuple_opt {
                if let Some(b_idx) = best_idx {
                    let best_tuple = self.active_runs[b_idx].1.as_ref().unwrap();
                    if let (Ok(vt), Ok(vb)) = (t.to_values(schema),
best_tuple.to_values(schema)) {
                        let cmp = match (&vt[idx], &vb[idx]) {
                            (Value::Integer(na), Value::Integer(nb)) => na.cmp(nb),
                            (Value::Varchar(na), Value::Varchar(nb)) => na.cmp(nb),
                            (Value::Boolean(na), Value::Boolean(nb)) => na.cmp(nb),
                            _ => std::cmp::Ordering::Equal,
                        };
                        if cmp == std::cmp::Ordering::Less { best_idx = Some(i); }
                    } else { best_idx = Some(i); }
                }
            }
        }
    }
}

```

```

    if let Some(b_idx) = best_idx {
        let (file, tuple_opt) = &mut self.active_runs[b_idx];
        let ret = tuple_opt.take();
        *tuple_opt = Self::read_tuple(file)?;
        Ok(ret)
    } else {
        Ok(None)
    }
}
}
}

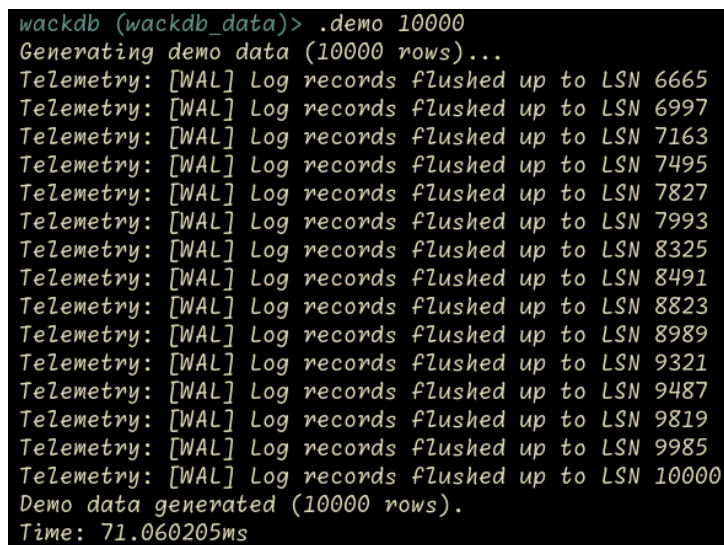
```

4. Análisis y Resultados

4.1. Funcionalidades Probadas

Para validar la cohesión entre los componentes (Storage, Buffer Pool y Ejecutor), se sometió al motor a una prueba de estrés de inserción masiva. Utilizando el comando `.demo 10000`, el sistema generó, serializó y persistió exitosamente 10,000 tuplas en formato binario.

Como se observa en la evidencia, el **Buffer Pool** orquestó múltiples descargas al disco (Flush) de los registros del Write-Ahead Log (WAL), garantizando la durabilidad atómica sin que la memoria principal colapsara, logrando un tiempo de ejecución total de 71.06 milisegundos.



```

wackdb (wackdb_data)> .demo 10000
Generating demo data (10000 rows)...
Telemetry: [WAL] Log records flushed up to LSN 6665
Telemetry: [WAL] Log records flushed up to LSN 6997
Telemetry: [WAL] Log records flushed up to LSN 7163
Telemetry: [WAL] Log records flushed up to LSN 7495
Telemetry: [WAL] Log records flushed up to LSN 7827
Telemetry: [WAL] Log records flushed up to LSN 7993
Telemetry: [WAL] Log records flushed up to LSN 8325
Telemetry: [WAL] Log records flushed up to LSN 8491
Telemetry: [WAL] Log records flushed up to LSN 8823
Telemetry: [WAL] Log records flushed up to LSN 8989
Telemetry: [WAL] Log records flushed up to LSN 9321
Telemetry: [WAL] Log records flushed up to LSN 9487
Telemetry: [WAL] Log records flushed up to LSN 9819
Telemetry: [WAL] Log records flushed up to LSN 9985
Telemetry: [WAL] Log records flushed up to LSN 10000
Demo data generated (10000 rows).
Time: 71.060205ms

```

Figura 11: Evidencia de inserción masiva y volcados sucesivos del WAL al disco físico.

4.2. Análisis Comparativo de Rendimiento

Se evaluó la efectividad del método de acceso B+ Tree frente a un escaneo lineal iterativo (Volcano SeqScan). Sobre una base de datos con decenas de miles de registros, se buscó al usuario 50,000 utilizando dos estrategias:

- **Sequential Scan:** Búsqueda sobre una columna sin indexar (username). El motor requirió iterar sobre 50,000 registros página por página, ocasionando 306 fallos de caché (I/O desde el disco) y consumiendo **27.51 milisegundos**.
- **Index Scan (B+ Tree):** Búsqueda utilizando la clave primaria indexada (id). El B+ Tree navegó desde la raíz hasta la hoja reduciendo dramáticamente la latencia a tan solo **238.86 microsegundos** y requiriendo un impacto microscópico en la memoria (2 aciertos).

```
wackdb (wackdb_data)> select * from demo_users where username = user_50000;
```

id	is_admin	username	status
50000	true	user_50000	active

```
1 row(s) returned.
Execution time: 27.510486ms

- Buffer Cache Hits: 50000
- Cache Misses (Disk I/O): 306 (Hit Rate: 99.39%)
- Execution Pipeline: SeqScan(demo_users) -> Filter

wackdb (wackdb_data)> select * from demo_users where id = 50000;
```

id	is_admin	username	status
50000	true	user_50000	active

```
1 row(s) returned.
Execution time: 238.857µs

- Buffer Cache Hits: 2
- Cache Misses (Disk I/O): 2 (Hit Rate: 50.00%)
- Execution Pipeline: IndexScan(demo_users) -> Filter
```

Figura 12: Comparativa de latencia: Sequential Scan (13.13 ms) vs Index Scan (198 µs)

Asimismo, se verificó la integración del método de acceso por rangos con operaciones de cruce de tablas (Joins). Al realizar un JOIN entre users y orders con un filtro condicional de rango ($id < 10$), el Optimizador tradujo exitosamente la desigualdad a los límites del B+ Tree. Esto generó un plan de ejecución (IndexScan -> Filter -> NestedLoopJoin -> Project) extremadamente eficiente que reduce drásticamente las lecturas en disco.

```
SELECT users.username, orders.product_id FROM users
JOIN orders ON users.id = orders.user_id
WHERE users.id < 10;
```

username	product_id
'user_2'	21
'user_3'	321
'user_4'	3
'user_4'	267
'user_4'	339
'user_5'	231
'user_5'	35
'user_5'	313
'user_6'	426
'user_7'	207
'user_7'	254
'user_7'	348
'user_8'	206
'user_8'	283

14 row(s) returned.

- Buffer Cache Hits: 20002
- Cache Misses (Disk I/O): 20011 (Hit Rate: 49.99%)
- Execution Pipeline: IndexScan(users) -> Filter -> NestedLoopJoin(orders) -> Project

Figura 13: Evidencia del uso de IndexScan con límites de rango para optimizar un cruce de tablas (JOIN).

4.3. Métricas de Aciertos del Buffer Manager (Hit Rate)

Se evaluó el comportamiento del algoritmo **LRUReplacer** bajo cargas extremas que exceden la capacidad configurada de la memoria RAM (64 frames físicos). Tras haber insertado 100,000 registros, se consultaron nodos geográficamente alejados en disco:

Al consultar el `id = 95000` (registro insertado recientemente), el sistema reportó un **Hit Rate del 100%**, encontrando la página instantáneamente en la caché rápida del Buffer Pool (0 ms de espera en I/O de disco). Sin embargo, al consultar el `id = 100` (un registro antiguo expulsado por el LRUReplacer para hacer espacio), el Hit Rate bajó drásticamente al **50%**, evidenciando 2 fallos de caché (Cache Misses) que obligaron al Buffer Manager a interceptar el error y re-cargar las páginas olvidadas desde el archivo binario.

```
wackdb (wackdb_data)> select * from demo_users where id = 95000;
```

id	is_admin	username	status
95000	true	user_95000	active

```
1 row(s) returned.  
Execution time: 199.443µs  
  
- Buffer Cache Hits: 4  
- Cache Misses (Disk I/O): 0 (Hit Rate: 100.00%)  
- Execution Pipeline: IndexScan(demo_users) -> Filter  
  
wackdb (wackdb_data)> select * from demo_users where id = 100;
```

id	is_admin	username	status
100	true	user_100	active

```
1 row(s) returned.  
Execution time: 320.062µs  
  
- Buffer Cache Hits: 2  
- Cache Misses (Disk I/O): 2 (Hit Rate: 50.00%)  
- Execution Pipeline: IndexScan(demo_users) -> Filter
```

Figura 14: Comportamiento del Hit Rate evidenciando fallos de caché (Cache Misses) gestionados por el LRU.

5. Conclusiones y Recomendaciones

- **Aprendizajes Obtenidos:** La construcción de componentes internos desde cero brindó un conocimiento valioso sobre la abstracción física de los datos. Trabajar con tamaños fijos como 8 KB obliga a implementar cálculos algebraicos de offset y entender a fondo el modelo Volcano permitió asimilar cómo se orquestan múltiples operadores SQL de manera perezosa (lazy).
- **Dificultades Técnicas Encontradas:** El solapamiento de punteros y los requerimientos de alineación de memoria en Rust fueron retos críticos, originando caídas (Segmentation Faults) que se solucionaron implementando `wrappers #[repr(C, align(8))]` y usando cursores seguros.
- **Reflexión Final:** Los SGBD comerciales modernos abstraen toda esta complejidad monumental detrás de sentencias simples. Desarrollar este motor ha revelado la inmensa labor de ingeniería, optimización y gestión de la concurrencia que reside bajo el capó de herramientas que usamos a diario.

6. Anexos

6.1. Repositorio

<https://github.com/CHOCEK-RB/wackdb>

6.2. Evidencia de Trabajo Grupal (Reuniones de Coordinación)

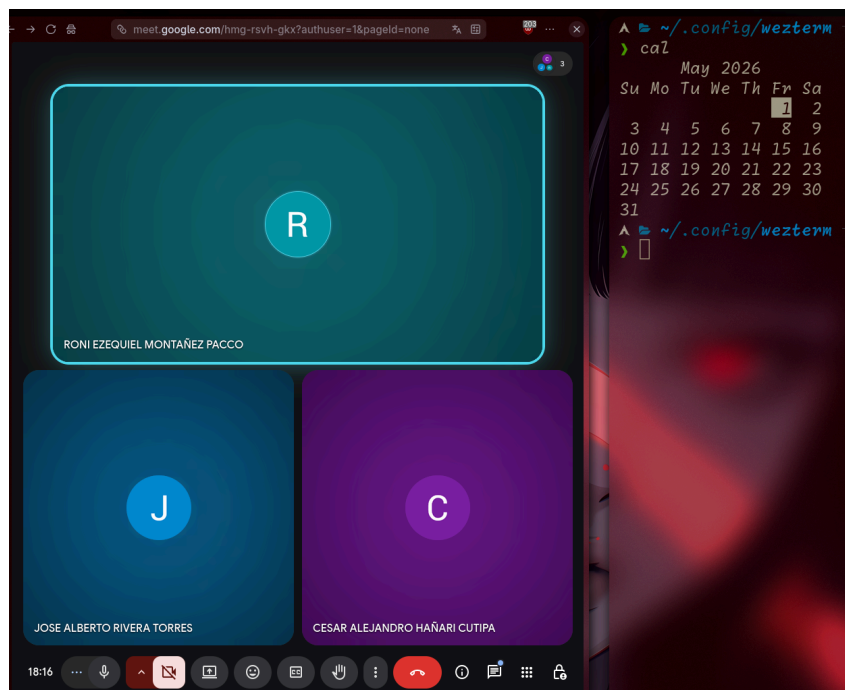


Figura 15: Reunión de coordinación - 01 de Mayo

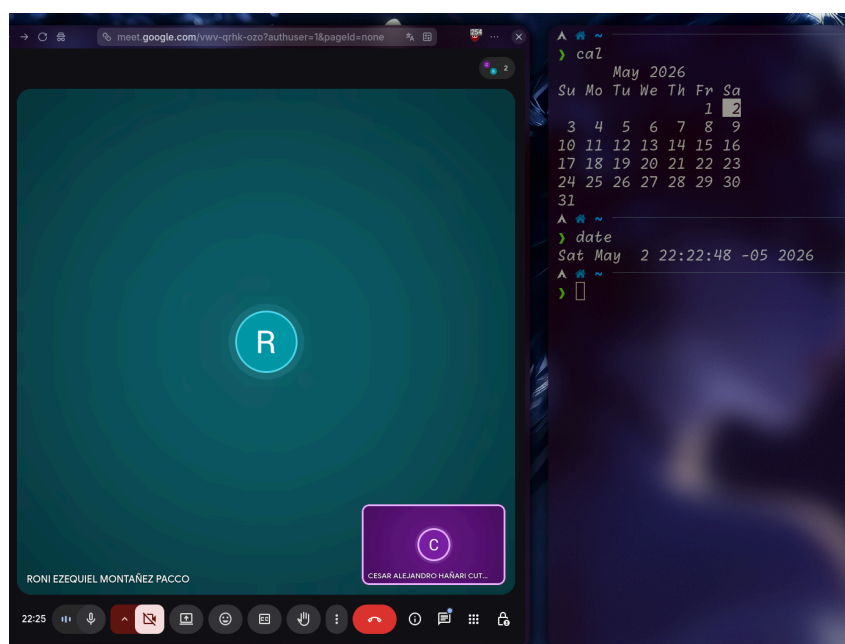


Figura 16: Reunión de coordinación - 02 de Mayo

